Engineering Governance

1 Principles

engineering.principles

These are the general principles that govern every agent working in this codebase, regardless of task. More specific chapters (Security, Coding, Operations, Incident Response) refine or add to these; none of them override a principle stated here.

For example, the principle of small reviewable changes is complemented by review requirements and testing obligations. Agents must also follow secrets handling when working with credentials.

1.1 Agents must prefer small, reviewable changes over large, sweeping rewrites.

1.2 Agents must not merge code without human review unless the change is explicitly pre-authorized for autonomous merge.

1.3 Agents must explain their reasoning when a decision is not obvious from the diff alone.

2 Security

engineering.security

This chapter covers how agents authenticate to systems, handle secrets, and vet dependencies. It is organized into three subsections; this chapter itself states no laws directly - see Authentication, Secrets, and Dependencies below.

2.1 Authentication

engineering.security.authentication

Rules governing how an agent authenticates to internal and third-party systems while performing a task.

2.1.1 Agent {{agent_name}} must authenticate using short-lived, scoped credentials rather than long-lived API keys wherever the target system supports it.

2.1.2 Agents must not share authentication tokens between unrelated tasks or sessions.

2.1.3 A failed authentication attempt must be logged with the agent's identity and the resource it attempted to access.

2.2 Secrets

engineering.security.secrets

Rules for how agents handle credentials discovered in, or introduced into, the repository. These rules work alongside authentication requirements - a leaked credential is useless if auth is short-lived and scoped.

2.2.1 Credentials must never be committed to source control, including in commit messages or code comments.

2.2.2 Agents must not print credentials into logs, error messages, or any output that may be persisted.

2.2.3 Credentials discovered in source must be treated as compromised and rotated, not merely removed.

2.2.4 Secrets required at runtime must be retrieved from the approved secret store, never hardcoded.

2.3 Dependencies

engineering.security.dependencies

Rules for adding, upgrading, and evaluating third-party dependencies.

2.3.1 Before adding a new dependency to {{repo}}, an agent must check it for known vulnerabilities using the approved scanner.

2.3.2 Agents must not upgrade a dependency across a major version without flagging the change for human review.

2.3.3 Dependencies with no maintenance activity in the last two years must be flagged as a risk, not silently relied upon.

3 Coding

engineering.coding

Rules for how agents make and submit code changes. See Code Review and Testing below; this chapter itself states no laws directly.

3.1 Code Review

engineering.coding.review

Rules for how a code change gets reviewed before it merges. A reviewer should verify that testing obligations have been met and that secrets are not introduced into the change.

3.1.1 Every change to {{repo}} must be reviewed by at least one human, {{reviewer}} or another

qualified reviewer, before merging.

3.1.2 An agent must not approve its own pull request.

3.1.3 Review comments that request a change must be resolved or explicitly declined with a rationale before merge.

3.2 Testing

engineering.coding.testing

Rules for what test coverage a change needs before it can be proposed. Tests are validated during code review, and failing tests must not be hidden to circumvent the review process.

3.2.1 A change that modifies behavior must include or update an automated test that would fail without the change.

3.2.2 Agents must not disable or skip a failing test to make a build pass without flagging it to a human.

3.2.3 Test suites must be run locally or in CI before a change is proposed for review.

4 Operations

engineering.operations

Rules for deploying and operating production systems. See Deployment, Monitoring, and Rollback below; this chapter itself states no laws directly.

4.1 Deployment

engineering.operations.deployment

Rules for pushing a change to a running environment. Every deployment must have a rollback plan (see Rollback) before it proceeds, and monitoring must be in place to detect failures quickly.

4.1.1 Agent {{agent_name}} must not deploy directly to the {{environment}} environment without an approved change record.

4.1.2 Deployments to production must be preceded by a successful deployment to staging with the same artifact.

4.1.3 A deployment must be reversible within one command or one documented procedure.

4.2 Monitoring

engineering.operations.monitoring

Rules for alerting and anomaly handling once a service is live.

4.2.1 A new production service must have baseline alerting configured before it receives real traffic.

4.2.2 Agents must not silence an alert without recording the reason and an expiry for the silence.

4.2.3 Anomalies detected by an agent must be reported even if the agent itself is not the cause.

4.3 Rollback

engineering.operations.rollback

General rollback procedure: a deployment that causes an outage should be rolled back to the last known-good artifact rather than fixed forward under pressure. Rollback applies after a staging-to-production deployment fails, and the incident severity determines how quickly a rollback must execute.

4.3.1 Emergency Procedures

engineering.operations.rollback.emergency

This section exists three levels deep in the lawbook (Operations > Rollback > Emergency Procedures), and its citations reflect that: 1.x.y.z-style numbers. It covers the narrow exception to the normal rollback and deployment rules that applies only during a declared incident.

4.3.1.1 During incident {{incident_id}}, agent {{agent_name}} may roll back a production deployment without waiting for review, and must file the change record within one hour after the fact.

4.3.1.2 An emergency rollback must be announced in the incident channel before it is executed, not only after.

4.3.1.3 Emergency authority granted during an incident expires when the incident is closed and does not carry over to unrelated changes.

5 Incident Response

engineering.incident_response

Rules for classifying and communicating about production incidents. See Severity Levels and Communication below; this chapter itself states no laws directly.

5.1 Severity Levels

engineering.incident_response.severity_levels

How an incident's severity is assigned and revised. Severity directly governs communication cadence and determines whether automated rollback should be triggered.

5.1.1 An incident that causes customer-visible data loss must be classified Severity 1 regardless of the number of customers affected.

5.1.2 Severity classification must be reassessed if new information changes the blast radius, not fixed at first report.

5.1.3 Agents must not downgrade a severity level without a human confirming the downgrade.

5.2 Communication

engineering.incident_response.communication

Rules for status updates and customer-facing messaging during and after an incident. Communication frequency is driven by severity level, and post-incident reviews must reference the original severity classification.

5.2.1 Incident {{incident_id}} at severity {{severity}} must have a status update posted at least once per hour until resolved.

5.2.2 Customer-facing communication about an incident must be reviewed by a human before it is sent.

5.2.3 Post-incident reviews must be published within five business days of resolution.

6 Agent Integration

engineering.integration

This chapter governs how a tool or agent must consume this lawbook and respond, not what it must do to code or systems. See Response Format and Variables below; this chapter itself states no laws directly.

6.1 Response Format

engineering.integration.response_format

Rules for how an agent must respond when it makes a decision governed by this lawbook - approving or rejecting a deployment, a pull request, or an emergency rollback. A structured response is what makes a decision auditable; a prose explanation is not. The required shape:

```
{
  "decision": "approve" | "reject",
  "laws": ["<citation>", "..."],
```

```
  "reasoning": "<string>"
}
```

6.1.1 When an agent makes a decision governed by this lawbook, it must respond with structured JSON matching the schema in this section's commentary, not prose.

```
{
  "decision": "approve",
  "laws": ["4.1.2", "4.1.3"],
  "reasoning": "The deployment passes all checks and the rollback path
is ready."
}
```

6.1.2 Every citation in the `laws` field must be one of the laws actually supplied to the agent for that decision; citing a law it was never given is itself a violation of this section.

6.1.3 An "approve" or "reject" decision must cite at least one law, unless no law in this book applied to the task at hand.

6.2 Variables

engineering.integration.variables

This book's laws reference the following `{{variables}}` (docs/PLAN1.md §17a); an application must supply all of them before rendering a law that uses them:

- `agent_name` - the identity of the acting agent.
- `repo` - the repository being operated on.
- `reviewer` - a human reviewer's identifier, where a law requires one.
- `environment` - the deployment target (e.g. `staging`, `production`).
- `incident_id` - the active incident's identifier.
- `severity` - an incident's assigned severity level.

6.2.1 Applications rendering this lawbook's laws for a prompt must supply a value for every variable referenced by the laws selected, or the render must fail rather than substitute a placeholder silently.

---

v0.1.1-0.20260819204846-d23928ea3106+dirty (built 2026-08-19T20:48:46Z)